기민수 KI MINSOO

SECURITY ENGINEER 정보보안 담당자 지원

메일 kiminsoo365@gmail.com
전화 010-4678-3603
GitHub github.com/ki0915
포트폴리오 ki0915.github.io/Profile/docs

미디어솔루션에서 1년 11개월 동안 대형병원, 대형 컨벤션센터, 5성급 호텔 고객사의 서버와 웹 서비스를 맡으며 침해사고 대응, 접근통제, 백업 설계를 직접 했습니다. 퇴사 후에는 정보보호학 학사와 컴퓨터보안 전문학사를 마치고 정보보안기사를 취득했으며, 생성형 AI를 쓸 때 개인정보가 새어 나가는 것을 막는 도구와 소스코드 취약점을 찾는 도구를 만들어 GitHub에 공개했습니다.

SKILLS 기술

보안 침해사고 대응, 이상행위 조사, IIS와 시스템 로그 분석, 접근통제(RBAC), 계정 정책, 방화벽과 Security Group 정책, 취약점 진단과 조치, DLP, SAST, SARIF, CWE, CVE

인프라 Windows Server 2022, IIS, Hyper-V, Ubuntu, AWS(EC2, RDS), Docker, SSL VPN, Grafana, Prometheus

개발 Python, TypeScript, Node.js, MSSQL, MariaDB, Redis, GitHub Actions

CERTIFICATIONS 자격

정보보안기사 2025.05
한국방송통신전파진흥원

CISA 시험 합격 (Associate) 2024.07
ISACA

CLLMSP (LLM 보안) 2026.07
Red Team Leaders

정보처리산업기사 2023.09
한국산업인력공단

Azure Fundamentals (AZ-900) 2022.08
Microsoft

네트워크관리사 2급 2021.11
ICQA

EDUCATION 학력과 교육

정보보호학 학사 2026.08
학점은행제

컴퓨터보안 전문학사 2026.02
명지전문대학

케이쉴드 주니어 (80시간) 2024.11
KISA

TOEIC Speaking IH (140) 2025.07
ETS

주요 경험 HIGHLIGHTS

침해사고 대응
고객사 웹 서버 침해사고 두 건에 현장 대응했습니다. IIS 로그와 앱 풀 프로세스를 대조해 루트킷 실행 경로를 찾았고, 서버를 AWS로 옮기면서 운영 IP만 허용했습니다.

병원 키오스크 CMS
병원 키오스크 약 200대를 원격으로 관리하는 CMS를 혼자 설계하고 개발했습니다. 병원 보안점검에 대응하면서 세션 검증 전에 권한이 먼저 붙던 문제를 찾아 고쳤습니다.

서버 통합과 백업
노후 서버 7대를 Hyper-V로 합치고, 스냅샷 하나뿐이던 백업을 변경이 제한된 백업 볼륨, AWS 이미지, MSSQL 복제로 나눴습니다. 퇴사 직후 랜섬웨어 사고 때 회사가 이 백업으로 복구했습니다.

컨테이너 장애 처리
배포 때 남은 Docker 이미지가 디스크를 채워 호텔 서비스가 멈췄을 때 정리하고, cron으로 자동화해 재발을 막았습니다.

DLP 프록시 개발
생성형 AI와 MCP 도구 호출에 섞여 나가는 주민등록번호, 카드번호, API 키를 찾아 가리거나 막는 Mcp-Guard를 만들어 GitHub에 공개했습니다.

AI 감사 도구 개발
한국 AI 기본법 요구사항 18개를 기준으로 AI 행동과 근거 문서를 대조하고, 근거가 부족하면 과제를 닫지 못하게 하는 EvidScope를 AI와 협업해 만들었습니다.

취약점 스캐너 개발
정적분석과 LLM으로 소스코드 취약점을 찾는 ETK-Scanner를 만들었고, 이 도구를 활용해 한 오픈소스의 인가 결함을 분석해 제보했습니다.

경력 EXPERIENCE

2023.04 – 2025.03
1년 11개월
연구개발팀 사원
(병역특례)

미디어솔루션

키오스크 CMS를 개발해 납품하고 고객사 서버 운영까지 맡는 회사입니다. 대형병원, 대형 컨벤션센터, 5성급 호텔 2곳의 서버와 웹 서비스를 담당했습니다.

고객사 웹 서버 침해사고 대응

사례 자세히 ↗

- 1차는 KISA 침해 통보로, 2차는 서비스 중단 신고로 알게 됐고 두 번 모두 현장에 나가 대응했습니다. 1차 때는 현장 시스템 접근과 관리자 계정부터 되찾고, IIS 로그와 앱 풀 프로세스를 대조해 루트킷 실행 경로를 찾았습니다.
- 2차 때는 CPU와 메모리가 과점유됐고 공격자가 깐 가상화 환경이 있어 크립토마이닝을 의심했습니다. ASP.NET 4.7.1을 공개 CVE와 대조해 취약한 구성을 찾았습니다.
- 서버는 고쳐 쓰지 않고 새로 세우기로 했고, 저는 이 결정을 도왔습니다. MSSQL 데이터는 격리 PC에서 무결성을 확인해 꺼냈고, AWS EC2와 RDS로 옮긴 뒤 Security Group은 운영 IP만 허용하고 RDP를 막았습니다.
- 사고 인지부터 복구 완료까지 4일이 걸렸고 퇴사할 때까지 같은 원인의 재발은 없었습니다. 기술 담당자와 경영진에게 원인과 비용을 따로 설명해 클라우드 이관과 정기 서버 관리에 합의했습니다.

병원 키오스크 CMS 개발과 보안점검 대응

사례 자세히 ↗

- 병원 키오스크 약 200대를 원격으로 관리하는 CMS를 혼자 설계하고 개발했습니다(TypeScript, Express, Angular, Redis, WebSocket). 원격으로 풀 수 있는 장애는 현장에 가지 않고 처리했습니다.
- 이 CMS가 병원 보안점검 대상이 되어 납품사로 수검에 대응했고, 모두 통과했습니다. 인증과 권한 구조를 고치면서 Redis 세션 검증이 끝나기 전에 임시 권한이 먼저 붙는 레이스 컨디션을 찾아 고쳤습니다.
- 관리자와 운영자 권한은 RBAC로 나눴고, JWT는 서버 개인키로 서명했습니다. 비밀번호 90일 변경, 재사용 금지, 로그인 실패 잠금도 넣었습니다. 망분리 내부망에서는 NICE 본인인증에 필요한 경로만 열었습니다.

서버 통합과 백업 재설계

사례 자세히 ↗

- 노후 서버 7대를 Windows Server 2022와 Hyper-V로 합치면서 인바운드를 기본 차단으로 바꾸고, 웹과 DB 서버에 외부에서 바로 원격 접속하던 경로를 없앴습니다.
- Hyper-V 스냅샷 하나뿐이던 백업을 변경이 제한된 별도 볼륨, AWS 서버 이미지, MSSQL 복제로 나누고 복구 절차서를 썼습니다. 퇴사 직후인 2025년 4월 랜섬웨어 사고 때 회사는 이 백업으로 데이터 손실 없이 하루 만에 복구했습니다.
- 팀원들이 사내 PC 이미지 갱신을 번거로워했습니다. 백업이 왜 필요한지, 그 이미지로 테스트 서버를 빨리 만들 수 있다는 점을 설명해 설득했습니다. 절차는 문서로 남겼습니다.

호텔 서버 운영과 장애 처리

사례 자세히 ↗

- 호텔 서버는 SSL VPN으로 접속해 관리자 페이지, DB, 서버 상태를 점검했습니다. Grafana와 Prometheus로 CPU, 메모리, 컨테이너 자원을 보다가 임계치에 닿기 전에 조치했습니다.
- 배포 때마다 남은 Docker 이미지와 중지된 컨테이너가 디스크를 채워 서비스가 멈췄을 때 원인을 확인해 정리하고 cron에 등록해 재발을 막았습니다. MariaDB 로그 장애도 해결했고, 서버 운영 체크리스트를 만들었습니다.

점검과 모의해킹 실습 LABS

보안 운영 랩: 월간 점검과 디스크 장애 재현

개인 랩, AI 협업 2026.10

점검 결과가 실제로 조치됐는지 달마다 추적하는 스크립트와, 실무에서 겪은 Docker 디스크 고갈 장애를 재현해 만든 디스크 정리 도구

Python, Bash, Docker, BuildKit, systemd timer, Prometheus

저장소 비공개 포트폴리오 소개 ↗

목적

점검 기준을 문서로 남기는 것만으로는 지적 항목이 실제로 고쳐졌는지 알기 어렵습니다. 실무에서 cron으로 막았던 디스크 장애도 다시 풀어 보려 했습니다. 이번에는 롤백용 이미지는 남기고 사용률에 따라 단계적으로 정리합니다.

만든 기능

- seccheck는 KISA Unix 계정과 권한 10개 항목, 로그인 실패가 몰린 IP, 허용 목록 밖 포트를 읽기 전용으로 점검합니다. 지난달 결과와 비교해 신규, 미조치, 조치 완료로 나누고 미조치 경과일을 셉니다.
- disk-guard는 사용 중이거나 보존 표시된 이미지, 롤백용 이미지는 지키면서 디스크 사용률 70%와 85%에서 단계적으로 정리합니다. 기본은 dry-run이고 정리 내역은 JSON 감사 로그로 남깁니다.

결과	**7/14 → 14/14** 1GiB 디스크에서 14번 배포할 때 성공 횟수. 서비스 쓰기 실패는 4회에서 0회	**91% → 38%** 단계적 정리에 BuildKit 빌드 캐시 상한을 더한 뒤 디스크 사용률
의의	점검 뒤 조치 여부까지 추적할 수 있게 됐고, 실무 장애를 재현해 정리 기준을 수치로 확인했습니다.	

허가된 실습 환경

TryHackMe 모의해킹 실습

Nmap으로 서비스를 열거했습니다. SMB 익명 접근과 IIS 공유 디렉터리 설정 오류가 이어지는 지점을 찾아 권한 상승까지 진행했고, 원인과 개선안을 보고서로 정리했습니다.

공개 문서
소개 페이지 ↗

Docker 컨테이너 보안 가이드

OWASP Docker Top 10, CIS Docker Benchmark, NIST SP 800-190을 읽고 10개 위험 항목을 위험도와 구현 비용 순으로 다시 묶어, 작은 팀이 먼저 할 일과 나중에 할 일을 나눴습니다.

AI 보안 프로젝트 PROJECTS

퇴사 후 만든 개인 프로젝트입니다.

Mcp-Guard 개인 프로젝트 2026.07

생성형 AI와 MCP 도구로 나가는 요청과 응답에서 개인정보와 기밀을 찾아 가리거나 막는 DLP 프록시

Python, HTTP 리버스 프록시, MCP stdio JSON-RPC, SSE, Docker, Helm github.com/ki0915/Mcp-Guard ↗ 포트폴리오 소개 ↗

목적

LLM API나 MCP 도구를 업무에 붙이면 주민등록번호, 카드번호, API 키가 프롬프트나 도구 호출에 섞여 외부로 나갈 수 있습니다. 네트워크 경계의 HTTP 프록시로는 MCP stdio 통신을 볼 수 없어서, 두 경로를 같은 정책으로 검사하게 만들었습니다.

만든 기능

- HTTP 리버스 프록시와 stdio JSON-RPC 어댑터를 만들어 MCP 도구 호출도 요청과 응답을 모두 검사합니다.
- 주민등록번호는 날짜와 체크섬, 카드번호는 Luhn과 IIN, API 키는 접두어와 엔트로피로 한 번 더 확인해 오탐을 줄였습니다. 항목마다 가림, 차단, 경보 중 하나를 고를 수 있습니다.
- 전각 숫자, 띄어 쓴 숫자, zero-width 문자, Base64로 감싼 값은 원문 위치를 유지한 채 풀어서 검사하고, 마스킹은 원문에 적용합니다.
- 검사 중 오류가 나면 통과시키지 않습니다(fail-closed). 감사 로그에는 원래 값 대신 탐지 종류와 규칙만 남깁니다.

결과	**16.7% → 93.3%** 합성 우회 표본 60건의 탐지율. 정상 표본 30건에서 오탐은 없었습니다	**p95 +3.65ms** 프록시를 거칠 때 늘어나는 요청 지연, 로컬 200회 측정
의의	MCP 도구 호출까지 한 정책으로 검사할 수 있게 됐고, 탐지율과 지연은 측정 범위와 함께 공개했습니다.	
다음 단계	여러 요청에 나눠 보내는 유출, 중첩된 Base64, WebSocket과 이미지, PDF 검사를 다루려 합니다. README에 한계로 적어 둔 항목입니다.	

ETK-Scanner 개인 프로젝트 2026.07

비용 상한 안에서 소스코드 취약점을 찾는 LLM 기반 SAST 도구

Python AST, JS/TS 패턴 분석, Claude API, SARIF 2.1.0, GitHub Actions github.com/ki0915/etk-scanner ↗ 포트폴리오 소개 ↗

목적

LLM 스캐너는 함수마다 비싼 모델을 여러 번 불러 패키지 하나에 수천 원이 듭니다. 개인이나 학생도 감당할 수 있는 비용에 맞춰 설계했습니다.

만든 기능

- 먼저 Python AST와 JS/TS 패턴으로 인증, 검증 함수 후보를 좁힙니다. 이 단계는 LLM을 쓰지 않아 PR마다 CI에서 돌립니다.
- 저비용 모델이 분류한 소수만 고성능 모델로 검증하고, 예산 상한에 닿으면 멈춥니다. 실행마다 단계별 비용을 cost.json으로 남깁니다.
- 결과를 CWE, 심각도, 파일 위치가 담긴 SARIF 2.1.0으로 GitHub Security 탭에 올리고, 심각도 high가 나오면 Actions job을 실패시킵니다.

결과

제보 1건
이 도구를 활용해 분석한 오픈소스 인가 결함. 메인테이너가 영향을 확인하고 수정하겠다고 회신했습니다

200 → 401
실습 백엔드에서 찾은 하드코딩 JWT 서명 키(CWE-798)로 위조한 토큰이 키를 옮긴 뒤 차단됨

121건 중 111건
SecurityEval 취약 코드 탐지(재현율 91.7%). 안전 표본이 없어 오탐률은 아직 재지 않았습니다

의의

LLM을 쓰지 않는 정적 단계는 PR마다 돌리고, LLM 검증은 정해 둔 예산 안에서만 쓰게 했습니다.

다음 단계

알려진 취약점을 OSV와 GHSA로 걸러 내고, tree-sitter로 JS/TS 오탐을 줄이고, 안전 표본을 붙여 오탐률을 재려고 합니다.

EvidScope

개인 프로젝트, AI 협업 — 2026.09 – 10

한국 AI 기본법 요구사항을 기준으로 AI가 한 일과 근거를 대조해, 사람이 충분한지 검토하도록 돕는 감사 증거 파일럿

Node.js 24, SQLite WAL, Ed25519 서명 원장, HMAC, OIDC, Docker, Kubernetes

github.com/ki0915/EvidScope ↗ 포트폴리오 소개 ↗

목적

실행 로그로는 AI가 실제로 무엇을 했는지, 그때 권한이 있었는지, 어떤 근거를 봤는지 설명하기 어렵습니다. 그래서 내부감사나 보안 운영 담당자가 AI 행동, 도구 결과, 당시 권한과 정책 버전, 원본 문서, 사람의 판단을 이어서 대조할 수 있게 했습니다.

만든 기능

- 한국 AI 기본법과 시행령의 제30~36조, 제40조를 요구사항 18개로 옮겼습니다. 생성형 결과 표시, 고영향 AI 판단, 사업자 책무(위험관리, 설명, 이용자 보호, 사람 감독, 문서 5년 보관), 영향평가, 국내대리인을 따로 검사합니다. 금융업이라는 이유만으로 고영향으로 정하지 않고, 법적 판단은 사람이 내리게 했습니다.
- 수집, 감사 화면, 저장(vault), 분석(worker)을 네 프로세스로 나누고 최종 인증과 원장은 vault만 갖게 했습니다. 출처는 HMAC과 nonce로 인증합니다. 원장은 Ed25519로 서명하고, 내보낸 증적은 별도 공개키로 따로 검증할 수 있습니다.
- 근거 문서는 암호화해 보관합니다. 해시와 정책 버전을 대조해 문서가 바뀌면 재검토로 돌리고, 근거가 부족한 과제는 종결하지 않습니다. 확인하지 못한 사실은 unknown으로 남깁니다.

결과

84 → 600건
요청마다 원장 전체를 다시 검증하던 병목을 고친 뒤, 합성 지속 부하 600건 중 접수 건수. p95는 약 6초에서 206.81ms

거부 30건
읽기 권한 감사자가 판정을 바꿀 수 있던 결함을 고친 뒤, 변경 요청 30건이 거부되고 서명 상태가 그대로인지 회귀 시험으로 고정

6건 중 6건
AI 대출, 신용평가를 쓰는 금융사 합성 사례. 정상 1건과 모델, 목적, 공급사 문서, 감독 계획 변경, 감독 기록 누락 5건에서 기대한 판정이 모두 나왔습니다

의의

감사 담당자가 AI 행동마다 근거를 따라가 확인할 수 있게 됐습니다. 무결성 검증은 그대로 두고 부하에서 요청이 밀리던 문제를 풀었습니다.

다음 단계

실제 조직의 IdP, 외부 SIEM과 연동하고 원격 KMS, WORM 저장소, 복구 목표를 검증하려 합니다.

범위

로컬 파일럿입니다. 실행 승인이나 차단, 법률 준수 인증, 실제 기관 연동은 하지 않습니다.